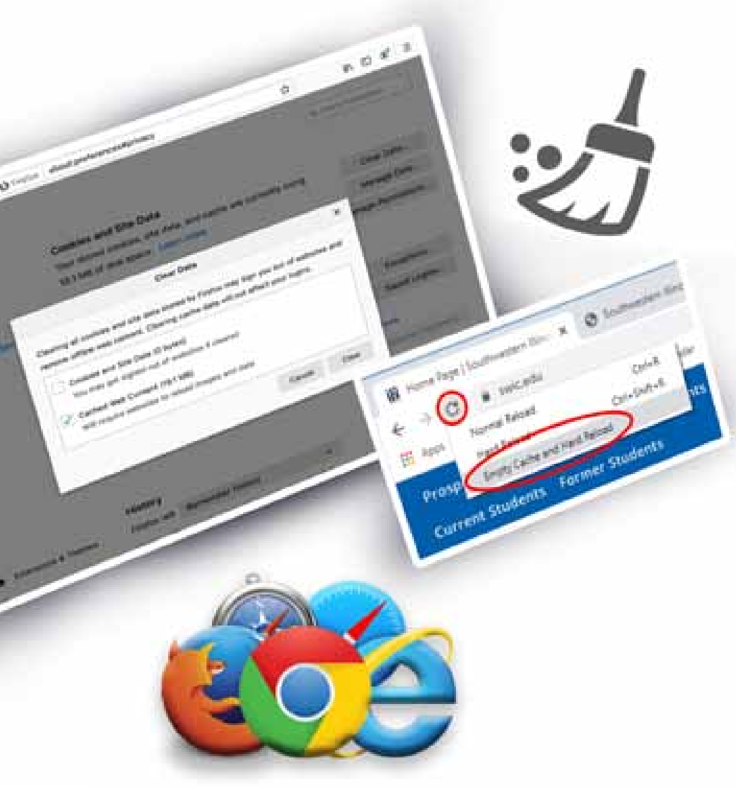
Understanding Cache Techniques

Cache is a crucial strategy for improving system performance as it temporarily stores data for quick retrieval, preventing the need to read from the original data source repeatedly, which can be time consuming. Several algorithms have been developed to increase the cache hit ratio. Some of these are discussed here.



In general, caching is used by operating systems, CPUs, GPUs, web browsers, applications, CDNs (content delivery networks), DNS (domain name systems), databases, and even at the ISP (internet service provider) level.

Before delving into a detailed discussion on cache, let's clarify some concepts often confused with it.

Caching vs buffering vs streaming vs register vs cookies

Caching: Storing partial or small data for quick retrieval
Buffering: Using a buffer to store data when there is a difference in the speed and processing of data between the sender and receiver
Streaming: Real-time data broadcasting, whether audio, video, or text
Register: Very small memory storage in computer processors for fast retrieval by the processor; typically stores data that the CPUs are processing
Cookies: Used in web browsers to maintain small data holding user preferences or login details, and is quite different from caching

We will see the various techniques used to save particular data temporarily, as we cannot save the entire stuff. Caching techniques can be used in different systems, such as:

- Cache memory
- Cache servers
- CPU cache
- Disk cache
- Flash cache
- Persistent cache

Cache is considered effective when the client requests data and it hits the cache rather than reading from the direct memory — this is termed as cache hit, else it's a cache miss. Several algorithms are used to increase the cache hit ratio based on application requirements. A few are briefly described below.

Cache algorithms

Spatial

- This caching technique is used to perform advanced reads of the nearest data from the recently used data.
- The idea behind reading closely associated data is it increases the chances of reading this data as well.
- This will increase the performance as the manual read is avoided and the data is ready to be served.
- For example, data saved in an array or similar type of records from a table can be read with a single read instruction from the original source and saved in the cache. This avoids multiple iterations of read requests.

FIFO (first in first out)

- Data is added to the queue as it is accessed.
- Once the cache is full, the first added item is removed from the cache.